

NeuroLog AI: demo guide

A practical guide to the repaired local application

1. Start and open the demo

This version opens directly to the dashboard. There is no login, sign-up, sign-out, or account/API-key setup in the interface. No Firebase project or external AI account is required for the presentation.

Install Node.js 22.12 or newer and Python 3.12 for a fresh checkout. The current workspace already has its dependencies installed. Open PowerShell in the project's root and run:

```
powershell -ExecutionPolicy Bypass -File .\start-preview.ps1
```

Open <http://127.0.0.1:5173/dashboard>. The launcher starts the services as hidden background processes, waits for the API to respond, and loads nine sample records if the database is empty. The services continue running after the launcher exits. Running the launcher again reuses services already listening on the expected ports.

If Python is not on PATH, pass its executable path with `-Python`. The launcher checks the installed dependencies and installs them only if missing. The app stores local data in `data/neurolog.db`. Logs for troubleshooting are written under `data/`.

2. A short presentation walkthrough

1. Show **Live Monitor**. It summarizes recent log severity, critical records, and incident markers. Sample data is already present when you use the launcher.
2. Click **Run clustering**. The actual Python TF-IDF/DBSCAN pipeline analyzes the latest 100 messages. At least five logs are required.
3. Open **Vector Analysis**. The initial sample batch has six repeated successful requests and three unusual messages. With that batch, the three unusual messages are outliers.
4. Open **Log Explorer**. Search for database, then choose **Errors / critical** to show syslog levels 0 through 3.
5. Open **AI Assistant**. Ask what to investigate, or choose **Analyze & Solve** next to a sample error. The local advisor returns a clearly labeled deterministic checklist when Groq is not configured.
6. Open **Demo Settings**. Change the color scheme, background, or visual effects for the presentation. It also has sample-loading and clustering controls.

Each **Load sample logs** click adds another nine records. Repeated copies of formerly unusual messages can form a cluster, so the number of outliers may change after adding another batch. The interface polls for new logs every two to five seconds.

3. What to explain to your audience

NeuroLog demonstrates how application messages can be collected, organized, and investigated. The data flow is:

Sample loader -> Flask API -> SQLite -> React dashboard.

Run clustering -> latest messages -> TF-IDF vectors -> DBSCAN groups -> saved outlier flags -> Vector Analysis.

TF-IDF represents messages by word importance. DBSCAN groups messages close in that numerical space, using `eps=0.5` and `min_samples=2`. Cluster -1 means a message was isolated from a group. An unusual message is a review candidate, not proof of a fault. Repeated failures can form a group, while a harmless unique message can be an outlier.

The recent-log health score is `max(0, 100 - 12 * critical_log_count)` over the latest 100 records. It is a simple severity heuristic, not a measured uptime percentage. The incident card reflects explicit markers supplied in log text; it does not independently verify remote service status or predict an actual crash.

The repository implements TF-IDF and DBSCAN. It does not contain an XGBoost or HDBSCAN model, a validated 94% accuracy result, or automatic remediation. The advisor never executes commands. Without a Groq key, its response is a deterministic demonstration of the advisory workflow rather than live LLM analysis.

4. Preview recovery and manual startup

If the preview becomes unavailable, run `start-preview.ps1` again and refresh the browser. The launcher was added because the earlier terminal-based preview stopped when its terminal processes ended.

For manual startup, keep two terminals open. In the root:

```
$env:NEUROLOG_DEMO='true'
.\.venv\Scripts\python.exe LogIntel_engine\api.py
```

In the other terminal:

```
cd logintel_ui
npm run dev
```

The backend listens on 127.0.0.1:5001. Vite listens on 127.0.0.1:5173 and proxies `/api` to the backend. Stop manually launched services with Ctrl+C. For background services, identify the command line in Task Manager and stop the project Vite and Python processes when the presentation is finished.

If startup fails, inspect `data/backend.stderr.log` and `data/frontend.stderr.log`. If a port belongs to a different application, stop that conflicting application before restarting. A backend already running in authenticated mode must be stopped so the launcher can start its local demo mode.

5. Files and verification

`LogIntel_engine/api.py` contains the supported API and clustering logic. `logintel_ui/src/pages` contains the screens. `src/api.js` sends requests through the local proxy. `start-preview.ps1` keeps the demo services running in the background. SQLite persists records between restarts. The legacy MongoDB examples and optional gateway are not required to present this demo.

Run `.\.venv\Scripts\python.exe -m pytest -q` from the root. In `login_tel_ui`, run `npm run lint`, `npm run build`, and `npm audit`. This demo-only update passed lint and build and was checked in the browser without login.

Keep this demo on the local machine; its UI intentionally has no authentication. Original project: <https://github.com/Sai-Nitin123/Neurolog-AI>. The original MIT license and contributor attribution are retained.